AutoRide: Real-Time Ride-Hailing, GeoFire & Concurrency Engine

Master Technical Handbook: Beginner to Extreme Senior Architecture

Project: AutoRide Ride-Hailing Platform
Engineer: Akshay Chandran P
Tech Stack: Flutter, Dart, BLoC, Firebase Realtime Database, GeoFire, Google Maps SDK, Razorpay, Concurrency Transactions

---

Section 1: End-to-End Architecture & Working Explanation

1.1 Beginner Level: How Ride Hailing Works

When a rider requests a cab: 1. The app detects current GPS coordinates. 2. The request queries nearby drivers within a 3-5 km radius. 3. Nearest drivers receive a push alert with a 15-second countdown to accept. 4. When a driver accepts, live location coordinates are streamed so the rider sees the car moving on the map in real time.

1.2 Extreme Senior Level: Firebase `runTransaction` Concurrency & Anti-Collision

When a ride is broadcast to 5 nearby drivers, two drivers might hit "Accept" at the exact same millisecond. If handled with standard writes, both drivers are told they got the ride, causing severe real-world conflict.
AutoRide Solution: Utilizes atomic database transactions via Firebase `runTransaction`:

```
Future<bool> acceptRide(String rideId, String driverId) async {
  final rideRef = FirebaseDatabase.instance.ref('rides/$rideId');
  final result = await rideRef.runTransaction((currentData) {
    if (currentData == null) {
      return Transaction.abort();
    }
    final map = Map<String, dynamic>.from(currentData as Map);
    // If already accepted by someone else, abort!
    if (map['status'] != 'SEARCHING') {
      return Transaction.abort();
    }
    map['status'] = 'ACCEPTED';
    map['driverId'] = driverId;
    map['acceptedAt'] = ServerValue.timestamp;
    return Transaction.success(map);
  });
  return result.committed;
}
```

1.3 Smooth Car Marker Animation & Bearing Interpolation

Raw GPS signals are noisy and update only once every 1 to 3 seconds, resulting in jerky teleportation of car markers on Google Maps.
AutoRide Solution: 1. **Spherical Linear Interpolation (Slerp):** Calculates intermediate positions between `Position A` and `Position B` over a 1000ms animation curve. 2. **Bearing Smoothing:** Calculates the mathematical vehicle bearing using the forward azimuth formula: $$ heta = ext{atan2}(\sin(\Delta\lambda)\cos(\phi_2), \cos(\phi_1)\sin(\phi_2) - \sin(\phi_1)\cos(\phi_2)\cos(\Delta\lambda))$$ Smoothly rotates the vehicle marker icon to match the road heading.

(Handbook continues with battery-optimized GPS tracking, GeoFire query mechanics, and Razorpay ride settlements)